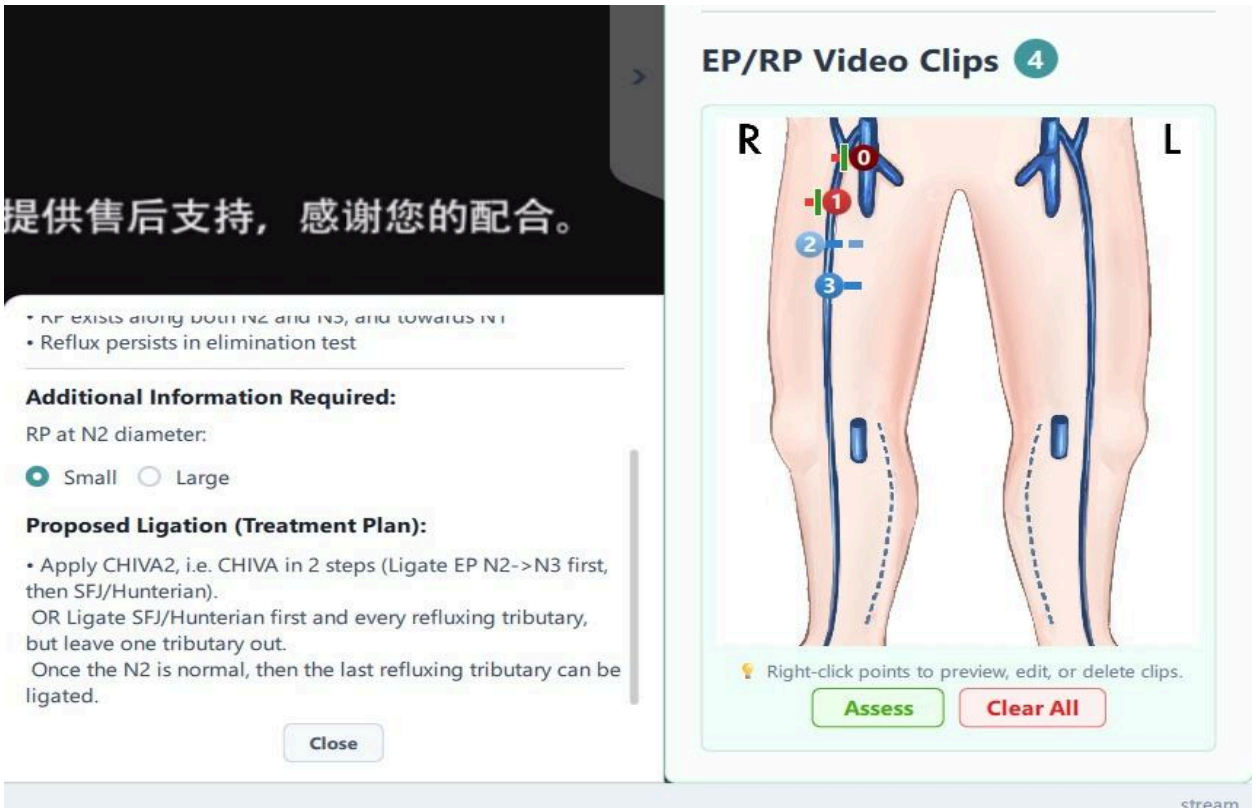
Shunt Analysis

Shunt Type 1:

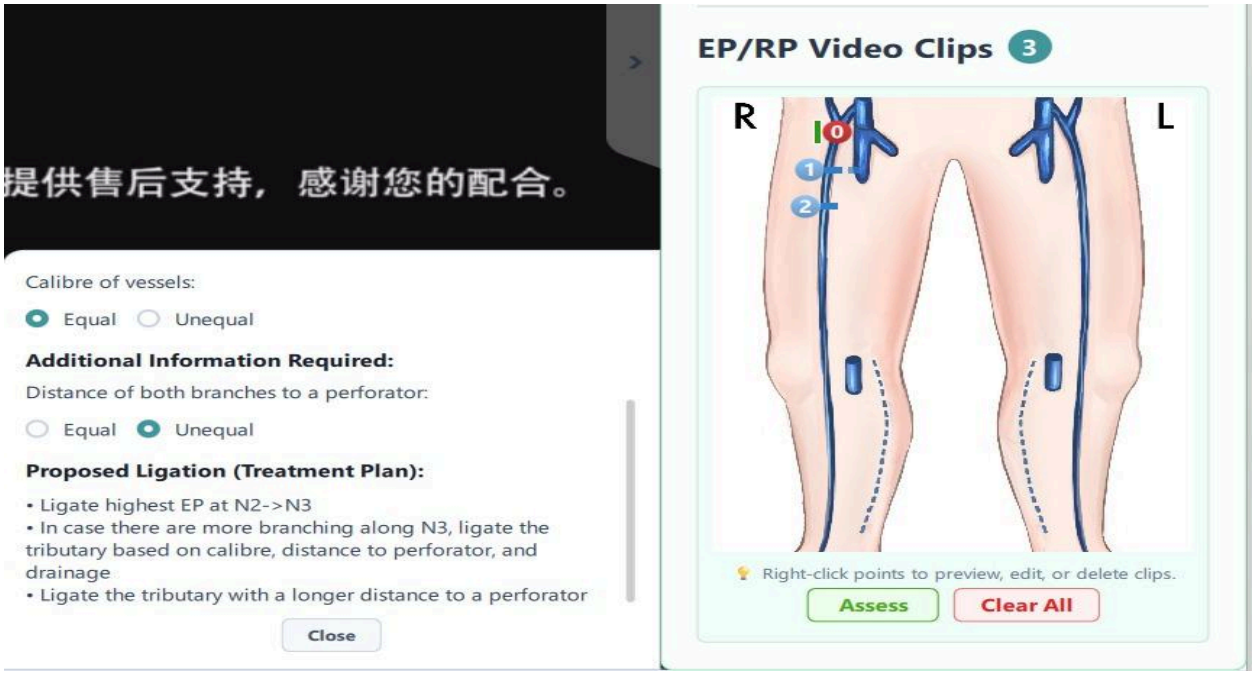
Shunt type 3 variant 1:

Shunt type 1+2 with small diameter :

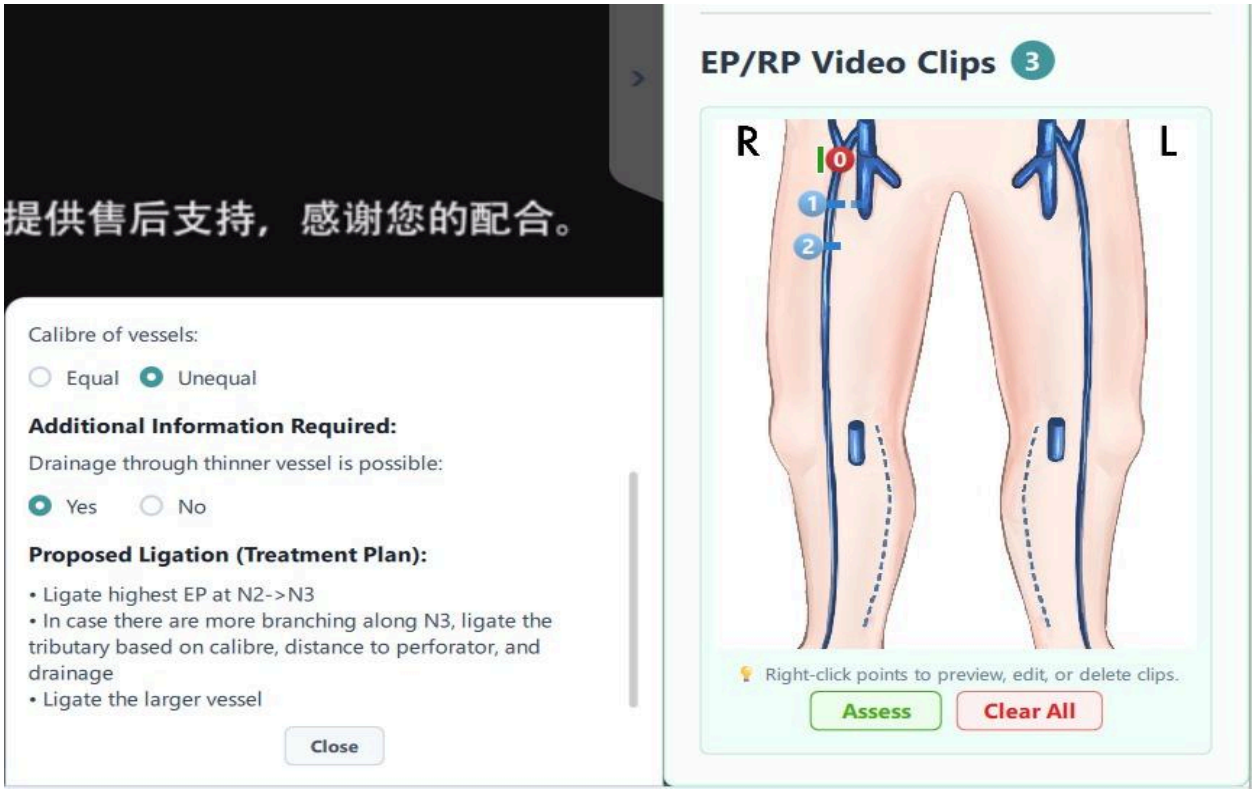
note: shunt type 1+2 has the same markers as shunt type 3, only difference is the RP N3->N1 marker has elimination test result as "Reflux" instead of "No Reflux".



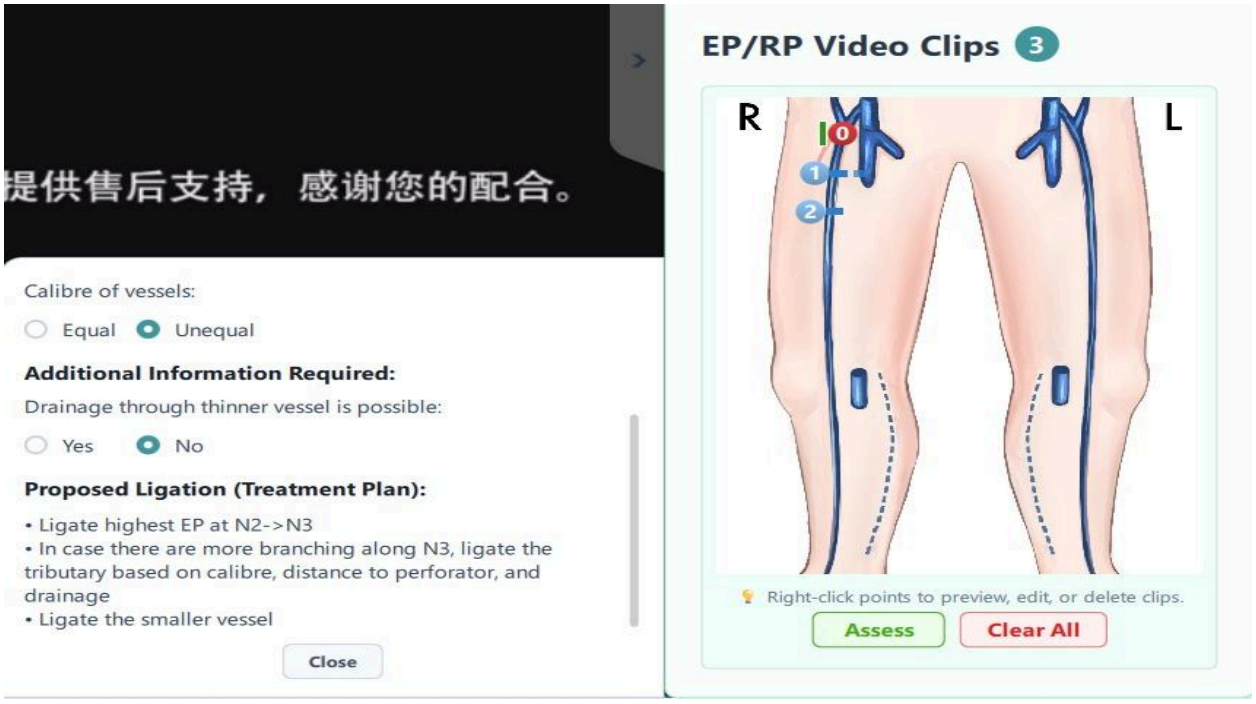
Shunt type 2 - equal calibre, unequal distance :



Shunt type 2 - equal calibre, yes to drainage :



Shunt type 2 - unequal calibre, no drainage :



Current Rule Based Code being used for Shunt Classification (Task-1) :

```
import logging
from typing import Any


class ShuntManager:
    """
    Manages ...
    """
    def __init__(self) -> None:
        self.logger = logging.getLogger(__name__)

    def classify_shunt(self, clip_list: list[dict[str, Any]]) -> dict:
        """Takes clipList from QML and returns:
        {
            "st1": bool,
            "st3": bool,
            "st12": bool,
            "needs_elim_test": bool,
            ...
        }
        """
        st1 = False
        st3 = False
        st12 = False
        st2a = False
        st2b = False
        st2c = False
        st_detected = True
        ask_diameter = False
        ask_aplastic = False
        ask_reflux_perforator = False
        ask_branching = False

        ep12 = False
        ep23 = False
        ep22 = False
        rp32 = False
        rp31 = False
        rp21 = False
        rp3 = False
        many_rp3 = False  # more than 1 RP at tributary
        many_rp21 = False
```

```
ep12_at_sfj = False
ep12_at_hunterian = False

elim_test_done = False
needs_elim_test = False
reflux_persists = False

sfj_min_x = 0.118
sfj_max_x = 0.473
sfj_min_y = 0.0
sfj_max_y = 0.098

hunterian_min_x = 0.118
hunterian_max_x = 0.473
hunterian_min_y = 0.098
hunterian_max_y = 0.353

reasoning = []
ligation = []

# list of ep12 to ligate
ep12_indexes = []
# list of rp21 to ligate, compare y position
rp21_indexes = []
# list of ep23 to ligate
ep23_indexes = []
# list of ep23 and ep22 to ligate for shunt type 2, compare y position
ep23_y_indexes = []

# Reset all ligation rectangles first in QML list
for clip in clip_list:
    if "ep_ligation_rect" in clip:
        del clip["ep_ligation_rect"]
    if "ep_ligation_rect2" in clip:
        del clip["ep_ligation_rect2"]

    if "rp_ligation_rect" in clip:
        del clip["rp_ligation_rect"]

# ============================================================
# === 1. PRE-SHUNT TYPE ASSESSMENT CLIPLIST CRITERIAS CHECK ===
# ============================================================

for idx, clip in enumerate(clip_list):
```

```
x = clip.get("posXRatio")
y = clip.get("posYRatio")
flow = clip.get("flow")
from_type = clip.get("fromType")
to_type = clip.get("toType")
elim_test_result = clip.get("eliminationTest")

# Check for EP at N1->N2
if flow == "EP" and from_type == "N1" and to_type == "N2":
    ep12 = True
    ep12_indexes.append(idx)
    # Check if there is EP N1->N2 at SFJ or Hunterian Perforator
    if (
        x is not None
        and y is not None
        and hunterian_min_x <= x <= hunterian_max_x
        and hunterian_min_y <= y <= hunterian_max_y
    ):
        ep12_at_hunterian = True
        ep12_at_sfj = False
    elif (
        x is not None
        and y is not None
        and sfj_min_x <= x <= sfj_max_x
        and sfj_min_y <= y <= sfj_max_y
    ):
        ep12_at_sfj = True
        ep12_at_hunterian = False

# Check for EP at N2->N3
if flow == "EP" and from_type == "N2" and to_type == "N3":
    ep23 = True
    ep23_indexes.append(idx)
    ep23_y_indexes.append((idx, y))
    # Check if elim test has been done
    if elim_test_result != "":
        elim_test_done = True
        if elim_test_result == "Reflux":
            reflux_persists = True
        elif elim_test_result == "No Reflux":
            reflux_persists = False

# Check for EP at N2->N2
if flow == "EP" and from_type == "N2" and to_type == "N2":
```

```
        ep22 = True
        ep23_y_indexes.append((idx, y))

    # Check for RP at N3->N2
    if flow == "RP" and from_type == "N3" and to_type == "N2":
        rp32 = True
    # Check for RP at N3->N1
    if flow == "RP" and from_type == "N3" and to_type == "N1":
        rp31 = True

    # Check for more than one RP at tributary
    if flow == "RP" and from_type == "N3":
        if rp3:
            many_rp3 = True
        else:
            rp3 = True

    # Check for RP at N2->N1
    if flow == "RP" and from_type == "N2" and to_type == "N1":
        rp21_indexes.append(
            (idx, y)
        )  # include y-axis position to ensure most distal RP does not get ligated
        if rp21:
            many_rp21 = True
        else:
            rp21 = True

    # Check elim test result at EP N2->N3
    if ep23 and elim_test_done:
        if elim_test_result == "Reflux":
            reflux_persists = True
        elif elim_test_result == "No Reflux":
            reflux_persists = False

# ============================================================
# 2. SHUNT TYPE DECISION TREE
# ============================================================
""" Determine shunt types based on collected data above """
if ep12_at_sfj or ep12_at_hunterian:
    if ep12_at_sfj:
        reasoning.append("• EP exists from N1 to N2 at SFJ \n")
    if ep12_at_hunterian:
        reasoning.append("• EP exists from N1 to N2 at Hunterian perforator \n")
```

```
# --- SHUNT TYPE 1 ---
if not ep23:
    reasoning.append("• No EP exists from N2 to N3 \n")
    if rp21 and not (rp31 or rp32):
        st1 = True
        reasoning.append(
            "• RP exists along N2, but no RP exists along N3 \n"
        )
    else:
        st_detected = False
        reasoning = []

# EP at N2->N3, and RP exists at N3 only, or RP exists at N2 only through N3->N2
elif ep23:
    reasoning.append("• EP exists from N2 to N3 \n")

    # --- SHUNT TYPE 3 ---
    if (rp32 or rp31) and (not rp21):
        st3 = True
        reasoning.append(
            "• RP exists along N3, but no RP exists along N2 \n"
        )

    elif rp32 and rp21:
        st3 = True
        reasoning.append("• RP exists along N2, and through N3 to N2 \n")

    # --- Elimination Test needed to tell between Type 3 and 1+2 ---
    # Requires thinking of how to add order between second type of Type 3 (2 RPs)
    elif rp31 and rp21:
        reasoning.append(
            "• RP exists along both N2 and N3, and towards N1 \n"
        )

        if not elim_test_done:
            needs_elim_test = True
            reasoning.append("• Elimination test missing. \n")
            ligation.append("• Elimination test missing. \n")

        # --- SHUNT TYPE 1+2 ---
        elif reflux_persists:
            st12 = True
            reasoning.append("• Reflux persists in elimination test \n")
```

```
            if not many_rp21 and many_rp3:
                reasoning.append(
                    "• There is one RP at N2 and more than one RP at tributary\n"
                )

            elif many_rp21 and many_rp3:
                reasoning.append(
                    "• There is more than one RP at N2 and more than one RP at tributary\n"
                )

        # --- SHUNT TYPE 3 ---
        elif not reflux_persists:
            st3 = True
            reasoning.append(
                "• Reflux does not persist in elimination test \n"
            )
        else:  # failsafe branch for unhandled logic
            st_detected = False  # type: ignore[unreachable]
            reasoning = []

    # fallback incase no shunt type detected but entered this branch
    else:
        st_detected = False
        reasoning = []

# --- SHUNT TYPE 2A ---
elif ep23 and not ep12:
    st2a = True
    reasoning.append(
        "• EP exists from N2 to N3, but no EP exists from N1 to N2 \n"
    )

elif ep22 and not ep12:
    reasoning.append("• EP starts at mid GSV from N2 to N2 \n")
    # --- SHUNT TYPE 2B ---
    if (rp32 or rp31) and (not rp21):
        st2b = True
        reasoning.append("• RP exists along N3, but no RP exists along N2 \n")
    # --- SHUNT TYPE 2C ---
    elif (rp32 or rp31) and rp21:
        st2c = True
        reasoning.append("• RP exists along both N2 (GSV) and N3 \n")
    else:
        st_detected = False
```

```
            reasoning.append(
                "• But neither Shunt Type 2B nor 2C because RP does not exist along N3 only or
along both N2 and N3 \n"
            )

        # If none of the shunt types are detected, clear reasoning
        else:
            st_detected = False
            reasoning = []

        # ============================================================
        # === 3. LIGATION DECISION TREE ===
        # ============================================================
        """ Append ligation text and assign rectangle markers to EP/RP that should be ligated """
        # Ligation for shunt type 1
        if st1:
            if ep12_at_sfj:
                ligation.append("• Ligation at the SFJ \n")

            elif ep12_at_hunterian:
                ligation.append(
                    "• Ligation at the Hunterian Perforator closer to N2 \n"
                )
            if many_rp21:
                ligation.append(
                    "• When appropriate, ligate below each RP along N2, except the most distal RP \n"
                )

                # Sort list of RP from N2 according to y-axis position, then remove the last one (most
distal)
                self._ligate_all_rp21_except_distal(clip_list, rp21_indexes)

            # Update clips with field for rectangle markings (EP12)
            for idx in ep12_indexes:
                self._mark_ligation_rect(clip_list, idx, "ep_ligation_rect")

        # Ligation for shunt type 1+2
        if st12:
            ligation = []
            ligation.append(
                "• Shunt Type 1+2 detected, requires more data to classify ligation"
            )

            if not many_rp21 and rp3:
```

```
            ask_diameter = True

        elif many_rp21 and rp3:
            ligation = []
            ligation.append(
                "• Ligate SFJ or Hunterian perforator, and every refluxing tributary\n "
                "• When appropriate, ligate below each RP along N2, except the most distal RP\n "
                "• If one of the RP at refluxing tributary is close to the N2, and that special RP is
dilated, ligate that special RP."
            )
            # Add rectangles to ligate all RP21 except most distal
            self._ligate_all_rp21_except_distal(clip_list, rp21_indexes)

        # Update clips with field for rectangle markings (EP23)
        for idx in ep23_indexes:
            self._mark_ligation_rect(clip_list, idx, "ep_ligation_rect2")

        # Update clips with field for rectangle markings (EP12)
        for idx in ep12_indexes:
            self._mark_ligation_rect(clip_list, idx, "ep_ligation_rect")

        # Not yet implemented: Special case where one of the RP32 is close to N2 (Fig 6.28d)

    # Ligation for shunt type 3
    if st3:
        ligation = []
        ask_aplastic = True  # Not yet implemented: Aplastic segment

        if not many_rp3:
            ligation.append("• Ligate the EP at N2->N3\n")
        elif many_rp3:
            ligation.append(
                "• Ligate every refluxing tributary where it joins the N2 (first step of CHIVA 2)\n"
            )

        if ep12_at_sfj:
            ligation.append(
                "• Follow up from 6 weeks to 6/12 months to assess reflux at proximal N2"
                "\n• If there is reflux, the SFJ must be treated/ligated\n"
            )
        if ep12_at_hunterian:
            ask_reflux_perforator = True

        # Update clips with field for rectangle markings (EP23)
```

```
        for idx in ep23_indexes:
            self._mark_ligation_rect(clip_list, idx, "ep_ligation_rect2")

        # Not yet implemented: Special case combined with Shunt Type 1 for now (Fig. 6.33)

    # Ligation for shunt type 2
    if st2a or st2b or st2c:
        ligation.append("• Ligate highest EP at N2->N3\n")

        if many_rp3:
            ligation.append(
                "• In case there are more branching along N3, ligate the tributary based on calibre,
distance to perforator, and drainage\n"
            )
            ask_branching = True

        # Mark highest EP23 with rectangle
        self._ligate_highest_ep23(clip_list, ep23_y_indexes)

    # Ligation for no shunt types detected
    if not st_detected:
        ligation = []

    return {
        "ST1": st1,
        "ST3": st3,
        "ST12": st12,
        "ST2A": st2a,
        "ST2B": st2b,
        "ST2C": st2c,
        "shuntDetected": st_detected,
        "elimTestDone": elim_test_done,
        "needsElimTest": needs_elim_test,
        "refluxPersists": reflux_persists,
        "EP12AtSFJ": ep12_at_sfj,
        "EP1AtHunterian": ep12_at_hunterian,
        "EP23": ep23,
        "RP32": rp32,
        "RP31": rp31,
        "RP21": rp21,
        "reasoning": reasoning,
        "ligation": ligation,
        "updatedClipList": clip_list,
        "askDiameter": ask_diameter,
```

```
        "askAplastic": ask_aplastic,
        "askRefluxPerforator": ask_reflux_perforator,
        "askBranching": ask_branching,
    }

def _ligate_all_rp21_except_distal(self, clip_list: list[dict[str, Any]], rp21_indexes: list) -> None:
    # Add rectangles to ligate all RP21 except most distal
    rp21_sorted = sorted(rp21_indexes, key=lambda t: t[1])
    rp21_to_ligate = rp21_sorted[:-1]  # all except last RP

    for idx, _ in rp21_to_ligate:
        self._mark_ligation_rect(clip_list, idx, "rp_ligation_rect")

def _ligate_highest_ep23(self, clip_list: list[dict[str, Any]], ep23_y_indexes: list) -> None:
    # For shunt type 2, add rectangle to ligate an EP23 or EP22 based on highest y position
    ep23_sorted = sorted(ep23_y_indexes, key=lambda t: t[1])
    highest_ep_idx = ep23_sorted[0][0]  # all except last RP

    self._mark_ligation_rect(clip_list, highest_ep_idx, "ep_ligation_rect2")

def _mark_ligation_rect(self, clip_list: list[dict[str, Any]], index: int, field_name: str) -> None:
    # Update clips with field for rectangle markings (e.g. EP23)
    original = clip_list[index]
    updated = original.copy()

    updated[field_name] = True
    clip_list[index] = updated
```